WILDTRACK CONSERVATION ALLIANCE

Wildlife Conservation Database

SQL Database Project Report

Schema design, SQL analytics, data integrity, performance and security, demonstrated through real-world conservation business scenarios.

4	10,000	75	MySQL
Core tables	Seed records	Business cases	Technology

PREPARED BY

R Hemant Kumar Patra

PROJECT TYPE

SQL database design and analytics

DATABASE

Project_2 (MySQL)

SCOPE

Academic and portfolio project

1. Executive Summary

WildTrack Conservation Alliance is presented as a multi-country wildlife conservation programme that protects endangered species across a network of forest reserves and national parks. Field rangers fit GPS tracking collars to individual animals, and the collars return location, speed, temperature and battery health. The database built in this project is central to patrol planning, equipment monitoring, anomaly detection, conservation analysis and reporting.

The solution is a MySQL relational database named **Project_2** with four connected tables: **Reserves, Species, Animals and GPSLogs**. It holds about 10,000 records, with GPS logs forming the majority. The project shows how SQL moves from basic retrieval to operational controls: aggregation, joins, subqueries, CTEs, window functions, date and string functions, constraints, views, indexes and user privileges.

4	10,000	75	3	3
Core tables	Seed records	Business cases	Reusable views	Indexes

Project Information

Item	Value
Project title	WildTrack Conservation Alliance: Wildlife Conservation Database
Project type	SQL database design, data analysis and real-world SQL problem solving
Prepared by	R Hemant Kumar Patra
Technology	MySQL / SQL (MySQL Workbench EER model)
Core tables	Reserves, Species, Animals, GPSLogs
Dataset size	15 + 25 + 200 + 9,760 = 10,000 valid seed records

Project Objectives

- Design a relational wildlife-conservation database with primary-key and foreign-key relationships.
- Protect data quality with NOT NULL, UNIQUE and CHECK constraints.
- Analyse GPS activity and collar battery health; identify inactive, low-reporting or problematic collars.
- Produce repeatable reporting with views and aggregation; speed up lookups with indexes.
- Demonstrate database security using users and privileges.

2. Business Scenario and Problem Statement

The scenario describes an operational database used by rangers, biologists, the data team and donor relations. It is not only an archive; it is a decision-support layer for field activity, equipment maintenance, research, management reporting and funding communication.

Wildlife monitoring

Animal and GPS records show where monitored animals are and how often they report.

Ranger operations

Counts, recent activity and unusual-speed analyses help prioritise field attention.

Collar maintenance

Battery readings and reporting gaps signal when to repair or replace a collar.

Research

Age, species, reserve and movement fields combine for population and behaviour studies.

Conservation planning

Species coverage, reserve coverage and conservation status support planning.

Reporting

Views and aggregate queries create repeatable outputs for management and donors.

Problem Statement

Wildlife conservation teams need a centralised relational database that manages reserves, species, individual animals and GPS readings while preserving data integrity. It must support operational questions such as inactive collars, low battery levels and unusual movement, and it must give different categories of user controlled access.

Scope of the Project

The project covers four core entities, a 10,000-record seed dataset, 75 business cases and database-administration examples. The scope includes DDL, DML, DQL, analytical SQL, data-quality controls, views, indexes and access control.

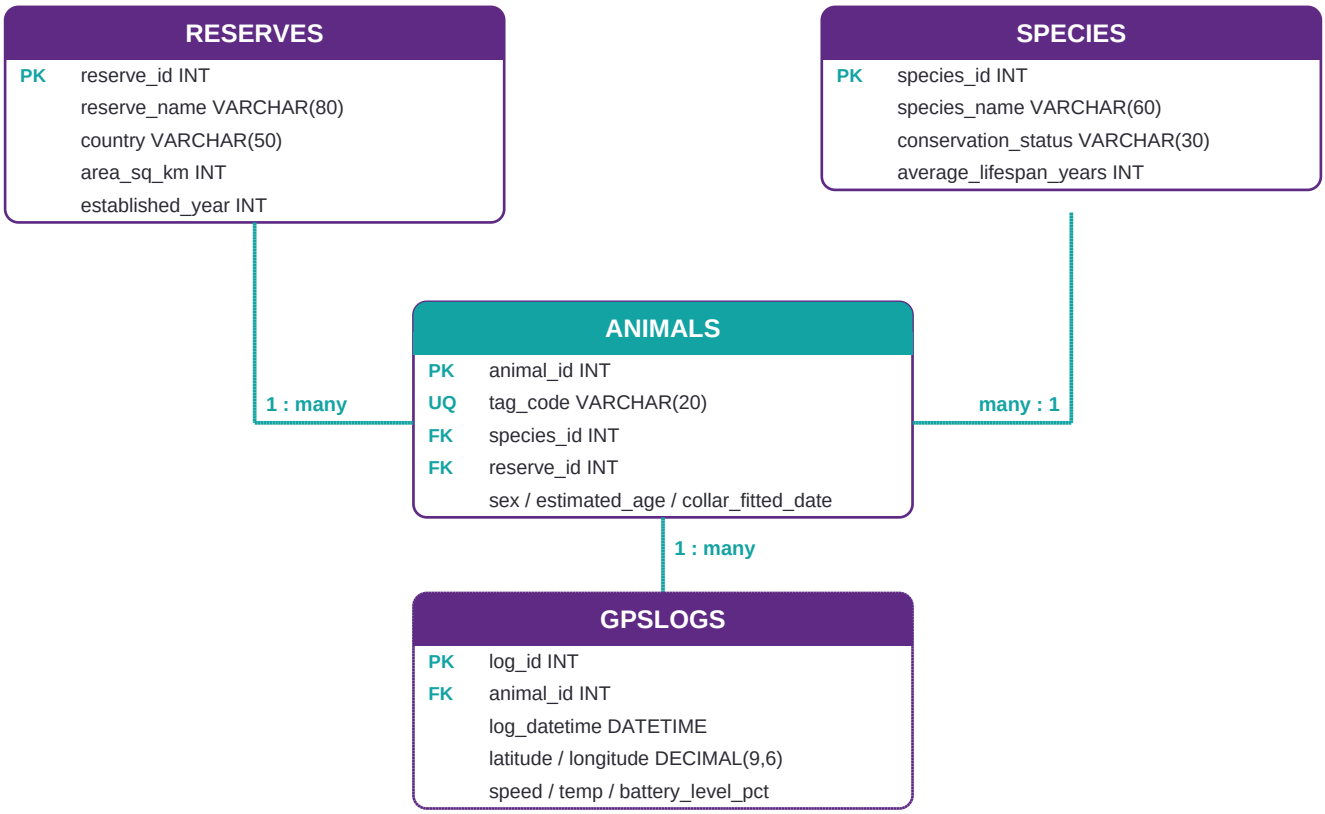
> **Scope boundary.** Python analytics, machine learning, dashboards and cloud deployment are treated as future improvements, not existing features.

3. SQL Concepts Covered

Area	Techniques used in the project
DDL / DML	CREATE DATABASE / TABLE / VIEW / INDEX, ALTER TABLE; INSERT, UPDATE, DELETE with JOIN
Querying	SELECT, WHERE, BETWEEN, IN / LIKE, date and time filters, NULL checks
Aggregation and joins	COUNT, COUNT(DISTINCT), AVG, MIN, MAX, GROUP BY, HAVING; INNER and LEFT JOIN
Subqueries / CTEs	Latest readings, highest and lowest values, multi-stage summaries
Window functions	Running counts, ROW_NUMBER, LAG, LEAD, DENSE_RANK, NTILE, rolling averages
Date, time, strings	DATE_SUB, DATEDIFF, TIMESTAMPDIFF, YEAR, QUARTER, DAYNAME, HOUR; UPPER, LENGTH, TRIM, CONCAT
CASE, views, indexes	Age bands, battery health, monitoring density; reusable views; composite indexes
Security	CREATE USER, GRANT, REVOKE, DROP USER, SHOW GRANTS

4. Database Architecture and EER Diagram

The architecture centres on **Animals**. Each animal references one species and one reserve through foreign keys, and GPSLogs references the animal. This separates relatively stable master data (reserves, species) from operational animal records and high-volume telemetry.



Relationship	Cardinality	Purpose
Reserves to Animals	1 : many	A reserve can hold many animal records.
Species to Animals	1 : many	A species can have many individual animals.
Animals to GPSLogs	1 : many	An animal can have many telemetry readings.

Why a Relational Database?

A relational model avoids storing reserve and species descriptions in every GPS record. Keys connect the entities, and analysts rebuild the business view with joins. The design also gives integrity rules a natural home: unique identifiers, required fields, valid numeric ranges and foreign-key relationships.

> **Architecture reading:** Reserves to Animals, Species to Animals, and Animals to GPSLogs. Master data stays separate from high-volume telemetry.

5. Table Design, Constraints and Data Quality

Seed data: **Reserves 15 | Species 25 | Animals 200 | GPSLogs 9,760** (total 10,000 rows).

Reserves

Column	Type	Constraints
reserve_id	INT	PRIMARY KEY
reserve_name	VARCHAR(80)	NOT NULL, UNIQUE
country	VARCHAR(50)	NOT NULL
area_sq_km	INT	CHECK (> 0)
established_year	INT	CHECK (>= 1900)

Species

Column	Type	Constraints
species_id	INT	PRIMARY KEY
species_name	VARCHAR(60)	NOT NULL, UNIQUE
conservation_status	VARCHAR(30)	NOT NULL
average_lifespan_years	INT	CHECK (> 0)

Animals

Column	Type	Constraints
animal_id	INT	PRIMARY KEY
tag_code	VARCHAR(20)	NOT NULL, UNIQUE
species_id	INT	FK to Species
reserve_id	INT	FK to Reserves
sex	VARCHAR(10)	CHECK IN (Male, Female, Unknown)
estimated_age	INT	CHECK (>= 0)
collar_fitted_date	DATE	NOT NULL

GPSLogs

Column	Type	Constraints
log_id	INT	PRIMARY KEY
animal_id	INT	FK to Animals
log_datetime	DATETIME	NOT NULL
latitude, longitude	DECIMAL(9,6)	none
speed_kmph	DECIMAL(5,2)	CHECK (>= 0)
temperature_c	DECIMAL(4,1)	none
battery_level_pct	INT	CHECK (0 to 100)

Data Quality and Validation

Risk	Database control / diagnostic	Cases
Invalid battery percentage	CHECK 0-100; query for out-of-range values	18
Negative age	CHECK estimated_age >= 0	58
Missing species	NOT NULL on Animals.species_id	51
Duplicate tag code	UNIQUE tag_code (already in schema)	39
Orphan GPS record	Foreign key + LEFT JOIN diagnostic	48, 64
Animals without logs	LEFT JOIN and NULL test	36, 70
Incorrect reserve assignment	Targeted UPDATE after validation	61
Whitespace errors	TRIM preview, then UPDATE	54

Why DECIMAL? Coordinates, speed and temperature use fixed-precision types to avoid floating-point issues in measured values.

6. Selected Business Cases

Eleven representative cases from the 75. Queries are the validated versions; each fix is noted under its case.

A. Joins, NULL Detection and Time Windows

CASE 02 Flag Animals with Fewer Than 5 Pings

GROUP BY | COUNT | CASE | LEFT JOIN

BUSINESS REQUIREMENT

Group GPS pings by animal and flag animals with fewer than 5 pings, since their collars may be malfunctioning.

```
SELECT a.animal_id,
       COUNT(g.log_id) AS total_pings,
       CASE WHEN COUNT(g.log_id) < 5 THEN 'Yes' ELSE 'No' END AS ping_status
FROM Animals a
LEFT JOIN GPSLogs g ON a.animal_id = g.animal_id
GROUP BY a.animal_id;
```

RESULT

Every animal has at least 31 GPS logs, so 0 animals fall below 5 pings.

VALUE

Highlights low-reporting collars that may need inspection.

Fix applied: the original INNER JOIN hid animals with zero logs; LEFT JOIN makes the rule complete.

CASE 03 Find Reserves with No Collared Animals

LEFT JOIN | NULL filtering

BUSINESS REQUIREMENT

Identify reserves that have no collared animal yet, a coverage gap worth flagging to the programme director.

```
SELECT r.reserve_id, r.reserve_name
FROM Reserves r
LEFT JOIN Animals a ON r.reserve_id = a.reserve_id
WHERE a.animal_id IS NULL;
```

RESULT

All 15 reserves have at least one collared animal.

VALUE

Reveals reserve coverage gaps for programme planning.

B. Aggregation, CTEs and Reporting

CASE 10 Species Population and Reserve Concentration

CTEs | GROUP BY | JOIN

BUSINESS REQUIREMENT

Tally each species' tracked population, then show which reserves host the heaviest concentrations of vulnerable species.

```
WITH SpeciesPopulation AS (
    SELECT species_id, COUNT(animal_id) AS total_tracked_population
    FROM Animals GROUP BY species_id
),
ReserveConcentration AS (
    SELECT species_id, reserve_id, COUNT(animal_id) AS reserve_animal_count
    FROM Animals GROUP BY species_id, reserve_id
)
SELECT s.species_name, s.conservation_status, r.reserve_name,
       sp.total_tracked_population, rc.reserve_animal_count
FROM ReserveConcentration rc
JOIN Species s ON rc.species_id = s.species_id
JOIN Reserves r ON rc.reserve_id = r.reserve_id
JOIN SpeciesPopulation sp ON rc.species_id = sp.species_id
WHERE s.conservation_status = 'Vulnerable'
ORDER BY rc.reserve_animal_count DESC;
```

RESULT

Lists vulnerable species by reserve, largest concentration first.

VALUE

Connects species vulnerability with geographic concentration.

CASE 42 Reserves with More Than 15 Animals

GROUP BY | HAVING

BUSINESS REQUIREMENT

Count animals in each reserve and shortlist reserves tracking more than 15 animals; they may need extra ranger support.

```
SELECT reserve_id, COUNT(*) AS animal_count
FROM Animals
GROUP BY reserve_id
HAVING COUNT(*) > 15;
```

RESULT

Reserves 4, 7 and 11 qualify, with 18, 17 and 17 animals.

VALUE

Identifies reserves that may need additional ranger support.

CASE 44 Consolidated Reserve Health Report

LEFT JOIN | aggregates | CASE

BUSINESS REQUIREMENT

One result per reserve: animal count, average age, average collar battery health and a battery-alert flag.

```
WITH AnimalSummary AS (
    SELECT animal_id, reserve_id, estimated_age FROM Animals
),
BatterySummary AS (
    SELECT animal_id, AVG(battery_level_pct) AS avg_battery
    FROM GPSLogs GROUP BY animal_id
)
SELECT r.reserve_id, r.reserve_name,
       COUNT(DISTINCT a.animal_id)        AS total_animals,
       ROUND(AVG(a.estimated_age), 2)     AS average_age,
       ROUND(AVG(bs.avg_battery), 2)      AS average_battery_level,
       CASE WHEN AVG(bs.avg_battery) < 20
            THEN 'Battery Alert' ELSE 'Healthy' END AS battery_status
FROM Reserves r
LEFT JOIN AnimalSummary a ON r.reserve_id = a.reserve_id
LEFT JOIN BatterySummary bs ON a.animal_id = bs.animal_id
GROUP BY r.reserve_id, r.reserve_name;
```

RESULT

One consolidated row per reserve for the annual conservation report.

VALUE

Provides a consolidated reserve health view.

Fix applied: battery is averaged per animal first; joining Animals straight to all GPS rows would repeat each animal's age once per GPS row.

C. Window Functions

CASE 07 Rolling Three-Reading Battery Average

Window function | ROWS frame

BUSINESS REQUIREMENT

For each reserve, smooth out sensor glitches by averaging the current battery reading with the two before it.

```
SELECT r.reserve_id, g.log_datetime, g.battery_level_pct,
       AVG(g.battery_level_pct) OVER (
           PARTITION BY r.reserve_id ORDER BY g.log_datetime
           ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS avg_battery_level
FROM Reserves r
JOIN Animals a ON r.reserve_id = a.reserve_id
JOIN GPSLogs g ON a.animal_id = g.animal_id
GROUP BY r.reserve_id, g.log_datetime, g.battery_level_pct;
```

RESULT

A smoothed battery trend per reserve over time.

VALUE

Reduces sensitivity to a single noisy battery reading.

CASE 23 Classify Battery Health

ROW_NUMBER | CASE

BUSINESS REQUIREMENT

Label each collar's current battery as Critical (below 15%), Low (15-40%) or Healthy (above 40%).

```
WITH LatestLogs AS (
    SELECT g.*,
           ROW_NUMBER() OVER (PARTITION BY animal_id
                              ORDER BY log_datetime DESC, log_id DESC) AS rn
    FROM GPSLogs g
)
SELECT animal_id, battery_level_pct,
       CASE WHEN battery_level_pct < 15 THEN 'Critical'
            WHEN battery_level_pct BETWEEN 15 AND 40 THEN 'Low'
            ELSE 'Healthy' END AS battery_level_status
FROM LatestLogs
WHERE rn = 1;
```

RESULT

One status per animal, based on its latest reading.

VALUE

Makes battery condition immediately clear to field teams.

Fix applied: the original classified every historical reading; the validated version first isolates the latest reading per animal.

CASE 40 Fastest Animal per Reserve

ROW_NUMBER | per-reserve ranking

BUSINESS REQUIREMENT

For every reserve, surface the single fastest recorded reading without dropping the reserve's other animals.

```
WITH Ranked AS (
    SELECT a.animal_id, a.reserve_id, g.log_id, g.speed_kmph,
           ROW_NUMBER() OVER (PARTITION BY a.reserve_id
                              ORDER BY g.speed_kmph DESC, g.log_id) AS rn
    FROM Animals a
    LEFT JOIN GPSLogs g ON a.animal_id = g.animal_id
)
SELECT animal_id, reserve_id, log_id, speed_kmph
FROM Ranked
WHERE rn = 1;
```

RESULT

Exactly one fastest reading per reserve.

VALUE

Finds the fastest recorded event in each reserve.

Fix applied: a windowed CASE could flag several rows per reserve; ROW_NUMBER guarantees one.

D. Integrity, Views, Indexing and Security

CASE 18 Prevent Invalid Battery Percentages

CHECK constraint

BUSINESS REQUIREMENT

Someone logged a battery level of 150% by mistake. What change makes this impossible going forward?

```
ALTER TABLE GPSLogs
ADD CONSTRAINT Check_battery_level
CHECK (battery_level_pct BETWEEN 0 AND 100);
```

RESULT

Values outside 0-100 are now rejected.

VALUE

Prevents impossible battery values from entering the database.

CASE 14 Reserve Battery Reporting View

CREATE VIEW | LEFT JOIN | aggregates

BUSINESS REQUIREMENT

Give the dashboard team a ready-made summary of each reserve's animal count and average battery health.

```
CREATE VIEW Reserve_Animal_Battery_level AS
SELECT r.reserve_id, r.reserve_name,
       COUNT(DISTINCT a.animal_id)  AS Animal_count,
       AVG(g.battery_level_pct)     AS Average_battery_health
FROM Reserves r
LEFT JOIN Animals a ON r.reserve_id = a.reserve_id
LEFT JOIN GPSLogs g ON a.animal_id = g.animal_id
GROUP BY r.reserve_id, r.reserve_name;
```

RESULT

Reporting tools can query the view directly.

VALUE

Creates a reusable management and reporting layer.

CASE 69 Read-Only Field Researcher Access

CREATE USER | GRANT SELECT

BUSINESS REQUIREMENT

A new field researcher may look at animal and GPS data but must never change it (least privilege).

```
CREATE USER 'Field_researcher'@'localhost' IDENTIFIED BY '<strong-password>';
GRANT SELECT ON Project_2.Animals  TO 'Field_researcher'@'localhost';
GRANT SELECT ON Project_2.GPSLogs  TO 'Field_researcher'@'localhost';
SHOW GRANTS FOR 'Field_researcher'@'localhost';
```

RESULT

The account can read Animals and GPSLogs only.

VALUE

Implements least-privilege, read-only access.

7. Security, Indexing and Views

Database Security

Requirement	SQL mechanism	Principle
Immediate withdrawal of access	DROP USER	Remove identity and access promptly
Read-only researcher	GRANT SELECT	Least privilege
Veterinarian updates	GRANT SELECT, UPDATE	Role-specific authority
Audit privileges	SHOW GRANTS	Verify effective permissions

Security observation. The original script contained example passwords directly in SQL. In production, credentials should be managed outside source-controlled scripts and rotated per policy. This report deliberately omits them.

Indexing and Views

Object	Definition	Purpose
Idx_gpslogs_animal_id	GPSLogs(animal_id)	Fast movement-history lookup by animal
Idx_Animals_reserve_animal	Animals(reserve_id, animal_id)	Reserve-to-animal access paths
idx_gpslogs_animal_date	GPSLogs(animal_id, log_datetime)	Animal plus chronological access
Reserve_Animal_Battery_level (view)	Case 14	Animal count and average battery per reserve
animal_latest_snapshot (view)	Case 55	Latest battery snapshot per animal
Critically_endangered_animals (view)	Case 56	Reusable donor-reporting list

8. Key Findings

All figures are calculated from the supplied seed data. Time-relative cases can produce different results if the database is later updated.

Metric	Dataset-derived result
Monitored species / tracked animals	25 species | 200 animals
Reserve coverage gaps	0 reserves without a collared animal
Animals with fewer than 5 pings	0
Lowest battery reading	3% (animal 148, GPS log 73)
Highest speed	70.00 km/h (animal 131, Jim Corbett National Park)
Overall average battery	51.08%
Reserves with more than 15 animals	Reserve 4: 18 | Reserve 7: 17 | Reserve 11: 17
Critically Endangered average age	12.95 years across 21 animals
Largest tracked species population	Striped Hyena, 16 animals
June 2026 GPS records	1,433
Nocturnal records (18:00-05:59)	4,813
Most-recorded weekday	Saturday, 1,475 GPS records

Rows per table (seed dataset)

GPS logs by movement class (Case 74 thresholds)

Resting, Moving and Fast Movement counts total 9,760 and match the GPSLogs row count. Day-time and night-time splits (4,813 nocturnal logs) support the nocturnal-behaviour study in Case 75.

9. Applications, Challenges and Conclusion

Challenges and Solutions

Challenge	SQL solution	Business benefit
Large GPS volume	Indexes on animal_id and animal/date	Faster movement-history queries
Duplicate identifiers	UNIQUE constraints	Cleaner animal registration
Invalid measurements	CHECK constraints	Impossible values rejected early
Missing relationships	FOREIGN KEY + orphan diagnostics	Protected referential integrity
Silent collars	LEFT JOIN + recent-log analysis	Finds animals needing inspection
Repeated reporting logic	Views	Consistent, reusable outputs
Complex analytics	CTEs + window functions	Readable multi-stage analysis
Multiple user types	GRANT / REVOKE / DROP USER	Role-appropriate access

Conclusion

The WildTrack project demonstrates practical SQL beyond isolated syntax exercises. A four-table relational structure gives a coherent model for reserves, species, animals and GPS telemetry, while the 75 cases force the analyst to choose techniques according to the business requirement. The strongest learning value is translating business language, such as *last seven days*, *fastest animal*, *below alliance average* or *read-only researcher*, into precise relational operations.

Future Improvements

- Power BI or Tableau dashboard connected to the curated views.
- Automated battery alerts and stored procedures for repeatable workflows.
- Real-time GPS monitoring, scheduled reports and automated anomaly detection.
- Python-based exploratory analysis and machine-learning models for behaviour or anomaly detection.
- Data-warehouse integration and cloud deployment with managed security.

Skills Demonstrated

Database	MySQL, relational design, EER modelling, primary and foreign keys
SQL	SELECT, JOIN, GROUP BY, HAVING, CASE, subqueries, CTEs, window functions, date and string functions
Data management	Cleaning, validation, integrity controls, targeted updates
Optimisation	Indexes, composite indexes, reusable views
Administration	Users, GRANT, REVOKE, DROP USER, SHOW GRANTS
Analytical thinking	Translating business requirements into SQL and interpreting operational significance